

AI Assisted Coding

Assignment 13.5

Name: M.PRASHANTH

Hall ticket no: 2303A51270

Batch no: 19

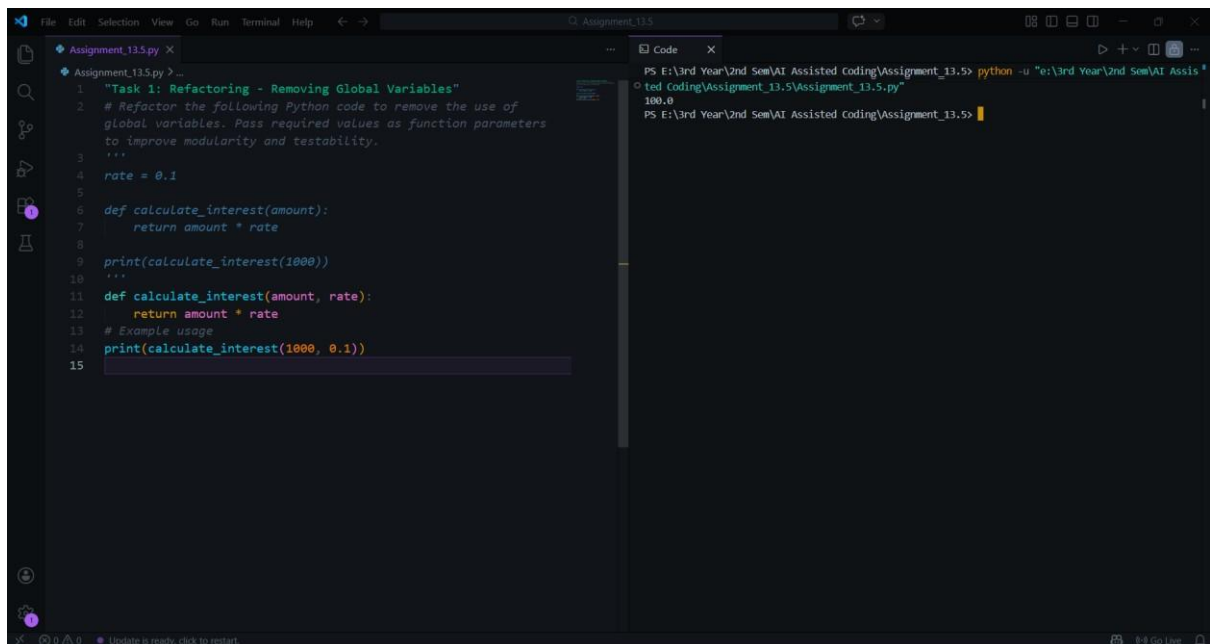
Task 1: Refactoring – Removing Global Variables

Prompt:

Refactor the following Python code to remove the use of global variables. Pass required values as function parameters to improve modularity and testability.

```
rate = 0.1
def calculate_interest(amount):
    return amount * rate
print(calculate_interest(1000))
```

Code & Output:



The screenshot shows a code editor with two panes. The left pane displays the refactored Python code, and the right pane shows the output of the code execution.

```
1 "Task 1: Refactoring - Removing Global Variables"
2 # Refactor the following Python code to remove the use of
3 # global variables. Pass required values as function parameters
4 # to improve modularity and testability.
5 """
6 rate = 0.1
7
8 def calculate_interest(amount):
9     return amount * rate
10
11 print(calculate_interest(1000))
12 """
13
14 def calculate_interest(amount, rate):
15     return amount * rate
16
17 # Example usage
18 print(calculate_interest(1000, 0.1))
```

The right pane shows the output of the code execution:

```
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_13.5> python -u "e:\3rd Year\2nd Sem\AI Assis
ted Coding\Assignment_13.5\Assignment_13.5.py"
100.0
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_13.5>
```

Explanation:

The original code used a global variable `rate`, which reduces modularity. The refactored version passes `rate` as a function parameter, making the function independent of global state. This improves testability and reusability.

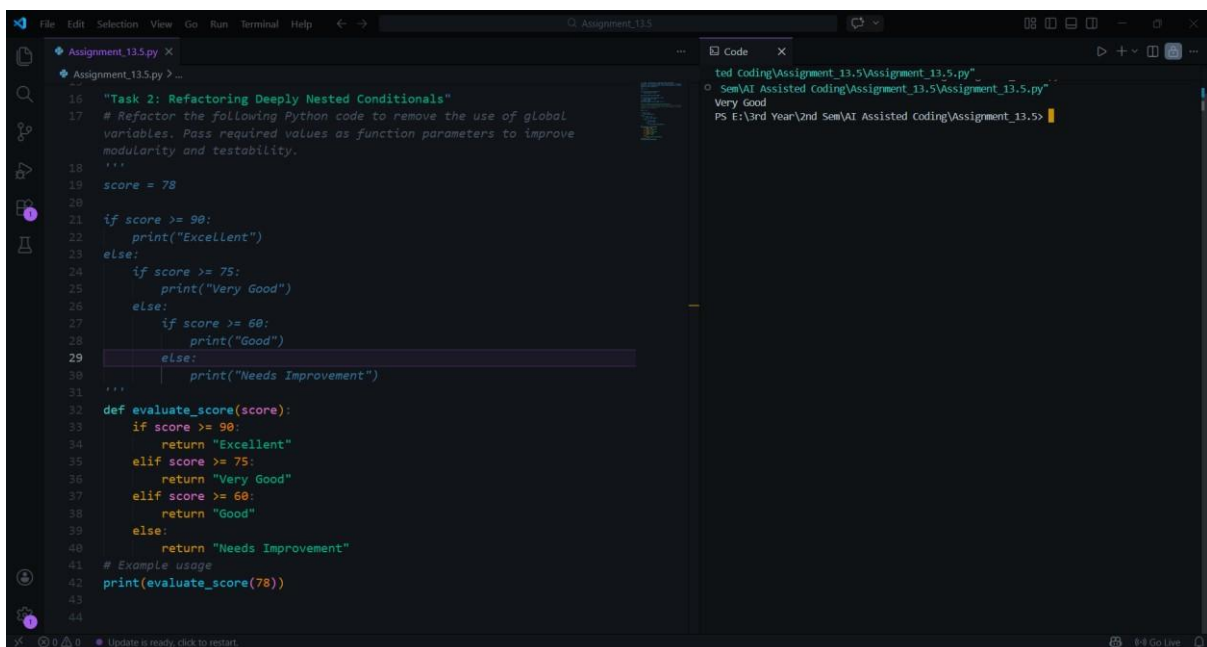
Task 2: Refactoring Deeply Nested Conditionals Prompt:

Refactor the nested conditional statements into a cleaner and more readable structure.

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Code

& Output:



Explanation:

The legacy code contains multiple nested if statements, which makes the control flow harder to read. The refactored version replaces nested conditions with a flat if-elif-else structure. This simplifies the logic and improves readability. It also follows standard Python coding practices for writing clear conditional statements.

Task 3: Refactoring Repeated File Handling Code

Prompt:

Refactor the repeated file open/read/close logic into a reusable function using context managers.

```
f = open("data1.txt")
print(f.read())
f.close() f =
open("data2.txt")
print(f.read())
f.close()
```

Code & Output:

```

44
45 "Task 3: Refactoring Repeated File Handling Code"
46
47 # Refactor the repeated file open/read/close logic into a reusable
48   function using context managers.
49   """
50   f = open("data1.txt")
51   print(f.read())
52   f.close()
53   f = open("data2.txt")
54   print(f.read())
55   f.close()
56   """
57   def read_file(file_name):
58       with open(file_name, 'r') as f:
59           return f.read()
60   # Example usage
61   print(read_file("data1.txt"))
62   print(read_file("data2.txt"))
63

```

```

Sem\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py
Sem\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py
Very Good
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd
Sem\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
This is a sample text file for Assignment 13.5.
Line 1: Introduction to AI Assisted Coding.
Line 2: Data files are used for input and output operations.
Line 3: Practice reading and writing files in Python.
Line 4: End of sample content.
Machine learning is a subset of artificial intelligence.
Neural networks are inspired by the human brain.
Data preprocessing is essential for accurate predictions.
Python is widely used for AI development.
Supervised and unsupervised learning are common paradigms.
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>

```

Explanation:

The original code repeatedly opens and closes files, which results in duplicated code. The refactored solution introduces a reusable function and uses the `with open()` context manager. This ensures that the file is automatically closed after use. It improves maintainability and follows the DRY principle.

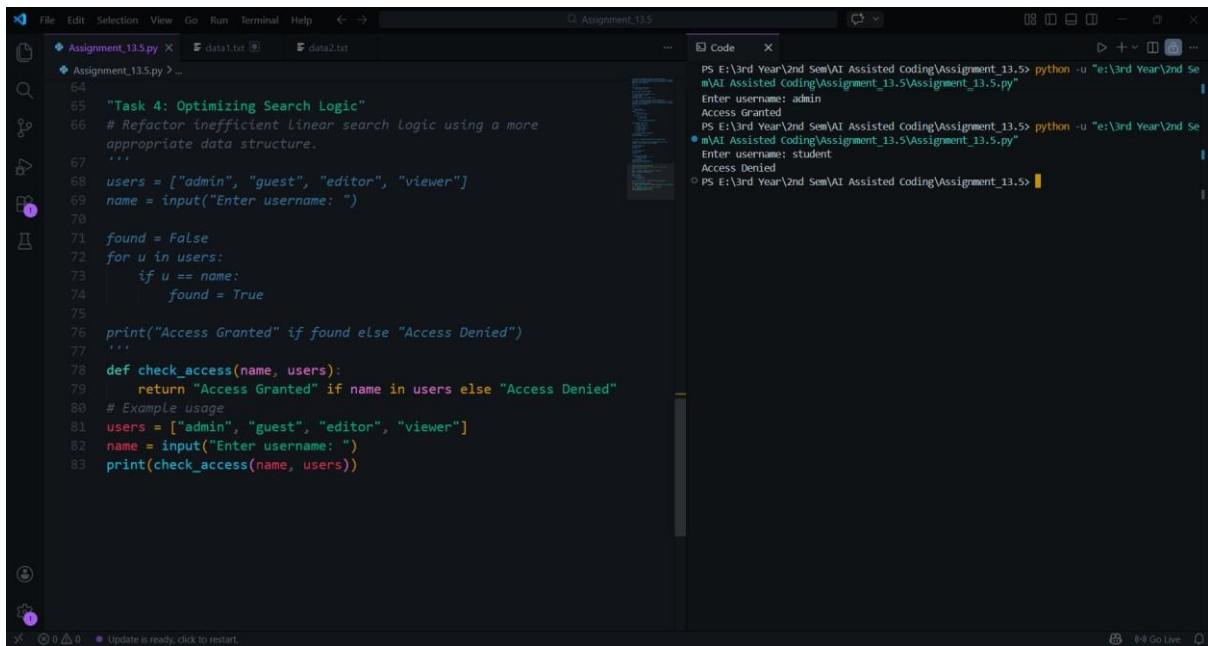
Task 4: Optimizing Search Logic Prompt:

Refactor inefficient linear search logic using a more appropriate data structure.

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
```

Code

& Output:



```
64
65 "Task 4: Optimizing Search Logic"
66 # Refactor inefficient linear search logic using a more
   appropriate data structure.
67 '''
68 users = ["admin", "guest", "editor", "viewer"]
69 name = input("Enter username: ")
70
71 found = False
72 for u in users:
73     if u == name:
74         found = True
75
76 print("Access Granted" if found else "Access Denied")
77 '''
78 def check_access(name, users):
79     return "Access Granted" if name in users else "Access Denied"
80 # Example usage
81 users = ["admin", "guest", "editor", "viewer"]
82 name = input("Enter username: ")
83 print(check_access(name, users))
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: admin
Access Granted
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: student
Access Denied
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>
```

Explanation:

The legacy code uses a loop to search through a list, which requires checking each element sequentially. This results in linear time complexity. The refactored version uses a set data structure, which allows constant-time membership checking. This improves performance and makes the code simpler.

Task 5: Refactoring Procedural Code into OOP Design

Prompt:

Refactor the following procedural salary calculation code into a class-based design.

```
salary = 50000 tax
= salary * 0.2 net
= salary - tax
print(net)
```

Code & Output:

```
84
85 "Task 5: Refactoring Procedural Code into OOP Design"
86 # Refactor the following procedural salary calculation code into
87   a class-based design.
88   """
89   salary = 50000
90   tax = salary * 0.2
91   net = salary - tax
92   print(net)
93   """
94
95   class SalaryCalculator:
96       def __init__(self, salary):
97           self.salary = salary
98
99       def calculate_tax(self):
100           return self.salary * 0.2
101
102       def calculate_net_salary(self):
103           tax = self.calculate_tax()
104           return self.salary - tax
105
106   # Example usage
107   calculator = SalaryCalculator(50000)
108   print(calculator.calculate_net_salary())
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: admin
Access Granted
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
Enter username: student
Access Denied
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5> python -u "e:\3rd Year\2nd Se
m\AI Assisted Coding\Assignment_13.5\Assignment_13.5.py"
40000.0
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5>
```

Explanation:

The legacy code performs calculations using simple procedural statements without structure. The refactored version introduces a class that encapsulates salary and tax calculations. This improves code organization and allows the logic to be reused easily. It also demonstrates object-oriented programming principles such as encapsulation.

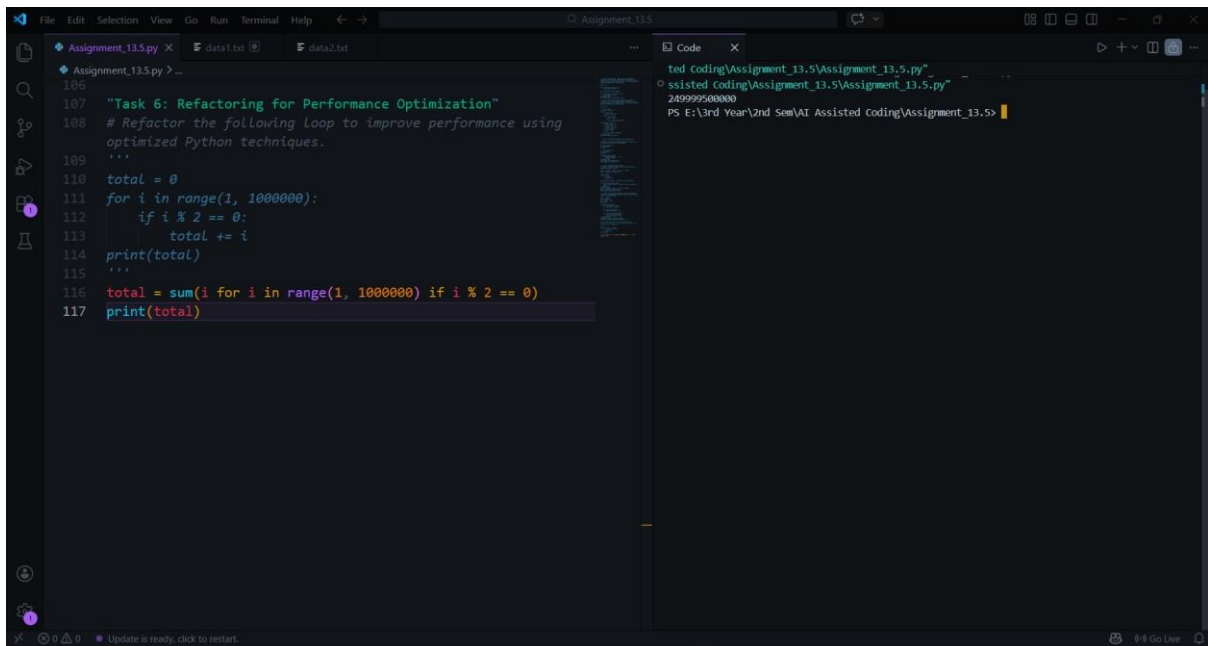
Task 6: Refactoring for Performance Optimization

Prompt:

Refactor the following loop to improve performance using optimized Python techniques.

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code & Output:



```
186
187 """Task 6: Refactoring for Performance Optimization"""
188 # Refactor the following loop to improve performance using
189 # optimized Python techniques.
190
191 total = 0
192 for i in range(1, 1000000):
193     if i % 2 == 0:
194         total += i
195 print(total)
196
197 """
198 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
199 print(total)
200 """
```

```
ted Coding\Assignment_13.5\Assignment_13.5.py"
o ssisted Coding\Assignment_13.5\Assignment_13.5.py"
2499999500000
PS E:\3rd Year\2nd Sem\AI\ Assisted Coding\Assignment_13.5>
```

Explanation:

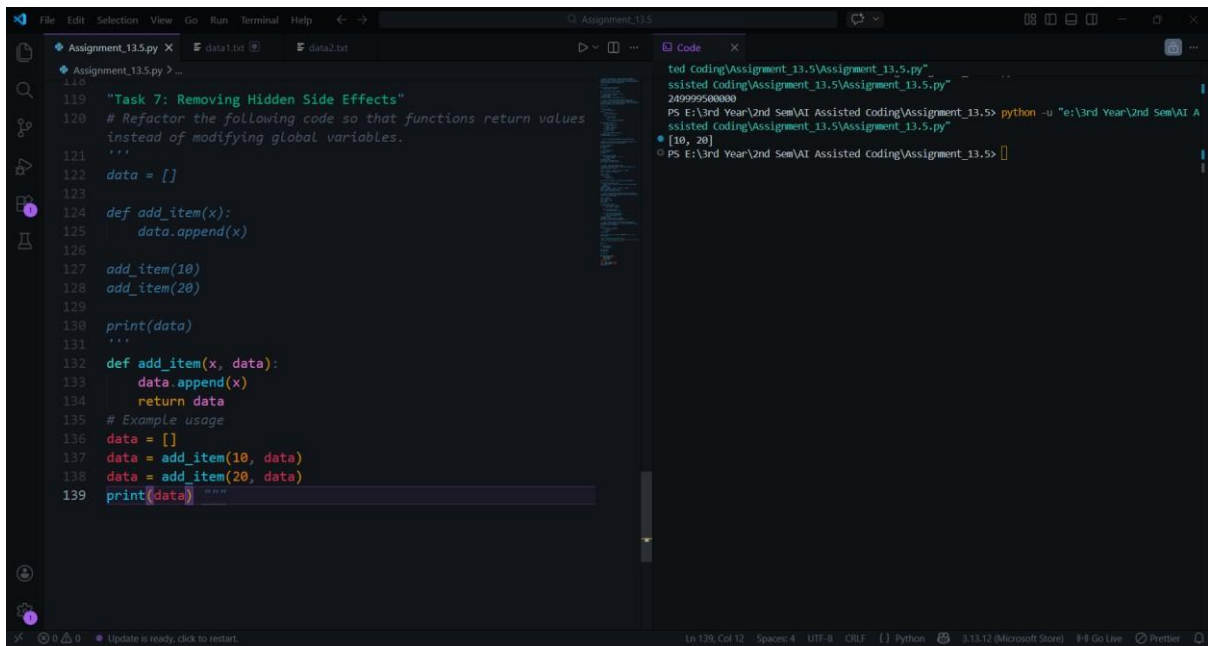
The original code manually iterates through a large range and checks each value. The refactored version uses Python's built-in `sum()` function with a generator expression. This approach reduces the number of lines and improves readability. It also takes advantage of optimized built-in functions.

Task 7: Removing Hidden Side Effects Prompt:

Refactor the following code so that functions return values instead of modifying global variables.

```
data = []
def add_item(x):
    data.append(x)
add_item(10)
add_item(20)
print(data)
```

Code & Output:



The screenshot shows a VS Code editor with a file named `Assignment_13.5.py`. The code is a Python script for 'Task 7: Removing Hidden Side Effects'. It shows a refactor of a function `add_item`. The original function (lines 121-129) modifies a global list `data`. The refactored function (lines 132-134) takes the list as an argument and returns a new list. The example usage (lines 136-139) demonstrates this by passing the current `data` list to `add_item` and then printing the result. A terminal window on the right shows the command `python -u "E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_13.5.py"` and the output `[10, 20]`.

```
119 "Task 7: Removing Hidden Side Effects"
120 # Refactor the following code so that functions return values
    instead of modifying global variables.
    """
121
122 data = []
123
124 def add_item(x):
125     data.append(x)
126
127 add_item(10)
128 add_item(20)
129 print(data)
    """
130
131
132 def add_item(x, data):
133     data.append(x)
134     return data
135
136 # Example usage
137 data = []
138 data = add_item(10, data)
139 data = add_item(20, data)
140 print(data)
```

Explanation:

The legacy function modifies a global list directly, which creates hidden side effects. This can lead to unexpected behavior in larger programs. The refactored version returns a new list instead of modifying global state. This functional approach improves predictability and makes the function easier to test.

Task 8: Refactoring Complex Input Validation Logic

Prompt:

Refactor the following password validation logic into modular validation functions.

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit()
for c in password):
        if any(c.isupper()
for c in password):
            print("Valid Password")
else:
    print("Must contain uppercase")
else:
    print("Must contain digit")
else:
    print("Password too short")
```

Code & Output:

```
141 "Task 8: Refactoring Complex Input Validation Logic"
142 # Refactor the following password validation logic into
143     modular validation functions.
144 '''
145 password = input("Enter password: ")
146
147 if len(password) >= 8:
148     if any(c.isdigit() for c in password):
149         if any(c.isupper() for c in password):
150             print("Valid Password")
151         else:
152             print("Must contain uppercase")
153     else:
154         print("Must contain digit")
155 else:
156     print("Password too short")
157 ...
158 def validate_password(password):
159     if len(password) < 8:
160         return "Password too short"
161     if not any(c.isdigit() for c in password):
162         return "Must contain digit"
163     if not any(c.isupper() for c in password):
164         return "Must contain uppercase"
165     return "Valid Password"
166
167 # Example usage
168 password = input("Enter password: ")
169 print(validate_password(password))
170
```

Explanation:

The original code contains multiple nested checks that make the logic harder to read. The refactored version separates each validation rule into its own function. This modular structure improves readability and makes testing each rule easier. It also allows new validation rules to be added without modifying the entire structure.

Final Conclusion:

This lab demonstrated how AI tools can assist in refactoring legacy Python code to improve readability, modularity, and performance. Techniques such as removing global variables, simplifying nested conditions, optimizing search logic, and applying object-oriented design were used. These improvements make the code easier to understand, maintain, and extend while preserving its original functionality.